北京郵電大學

数据结构课程设计

姓　名 李韶庸

学　院 国际学院

专　业 智能科学与技术

班　级 2024215122

学　号 2024213672

班内序号 17

指导教师 _____刘咏彬_____

2026 年 03 月

目录

一、实验题目.....3
二、题目分析与算法设计.....3
2.1 问题理解.....3
2.2 算法设计.....3
2.3 问题发现与解决.....5
三、程序复杂度分析.....6
3.1 时间复杂度.....6
3.2 空间复杂度.....6
四、程序运行结果展示.....6
4.1 最短路径查询.....6
4.2 全建筑遍历.....7
4.3 建筑检索.....7
4.4 非法输入校验.....8
五、代码提交说明.....9
六、实验过程记录.....9
七、心得与总结.....10
7.1 算法理解的深化.....10
7.2 数据结构选择的重要性.....11
7.3 系统设计的全局思维.....11
7.4 创新点.....11

数据结构课程设计

一、实验题目：北邮本部校园导航系统

设计并实现一个基于图结构的校园导航系统，支持建筑检索、任意两点间最短路径查询、全校园建筑遍历规划，以及交互式地图可视化展示。系统数据来源于北邮本部真实校园地图，包含 114 个节点（建筑与路口）、147 条道路。

二、题目分析与算法设计

2.1 问题理解

本题本质是一个加权无向图上的多功能路径查询问题，具体包含三个子问题：

2.1.1 最短路径查询

给定起点 s 和终点 t，求图中从 s 到 t 的最短路径及其长度。这是经典的单源最短路径问题。

2.1.2 全建筑遍历规划

给定出发点，规划一条依次经过所有建筑（非路口节点）的访问顺序，使总行程尽可能短。本质上是旅行商问题（TSP）的近似版本，图中节点间并非直接相连，需结合最短路算法使用。

2.1.3 建筑检索

根据关键字在建筑名称和描述中进行多级模糊匹配，快速返回候选建筑列表。

数据规模：

项目	数量
节点总数 V	114（其中路口 31 个，建筑 83 个）
边总数 E	147
需遍历建筑数 N	83

2.2 算法设计

2.2.1 图的存储结构

采用邻接表表示图，使用 Map<String, List<Edge>> 实现，以建筑名称字符串作为节点标识符。与邻接矩阵相比，邻接表在稀疏图（本题 $E=147$ 远小于 $V^2=12996$）下空间利用率更高，遍历邻居效率也更优。

```
static class Edge {
    String target;
    int weight;
}
Map<String, List<Edge>> adjList = new HashMap<>();
```

2.2.2 Dijkstra 最短路径

使用优先队列优化的 Dijkstra 算法求单源最短路径：

1. 初始化：起点距离为 0，其余节点距离为正无穷
2. 每次从优先队列中取出当前距离最小的节点 u
3. 松弛 u 的所有邻边：若 dist[u] + w(u,v) < dist[v]，更新 dist[v]并记录前驱 prev[v]=u
4. 重复直到队列为空，通过 prev 数组回溯重建完整路径

时间复杂度：$O\left((V+E)\log V\right)$

```
PriorityQueue<String> pq = new PriorityQueue<> (
    Comparator.comparingInt(n -> dist.getOrDefault(n, Integer.MAX_VALUE))
);
```

2.2.3 最近邻贪心遍历

全建筑遍历问题是 NP-Hard 的 TSP 问题，采用最近邻贪心策略给出近似解：

1. 从起点出发，将所有非路口建筑加入未访问集合

2. 每步对当前位置查询距离最近的未访问建筑
3. 移动到该建筑，标记为已访问，重复直至全部访问完毕

优化：距离矩阵预计算缓存

- 朴素实现中每步重新运行 Dijkstra，共需运行 $N=83$ 次，总时间 $O(N\cdot(V+E)\log V)$。
- 优化方案：
  - 首次调用遍历时，对所有 $N=83$ 个节点预计算 Dijkstra 并缓存到距离矩阵。
  - 此后每次调用直接 $O(1)$ 查表，总遍历开销降至 $O(N^2)$。
  - 当图结构发生变更时，自动使缓存失效并在下次查询时重建。

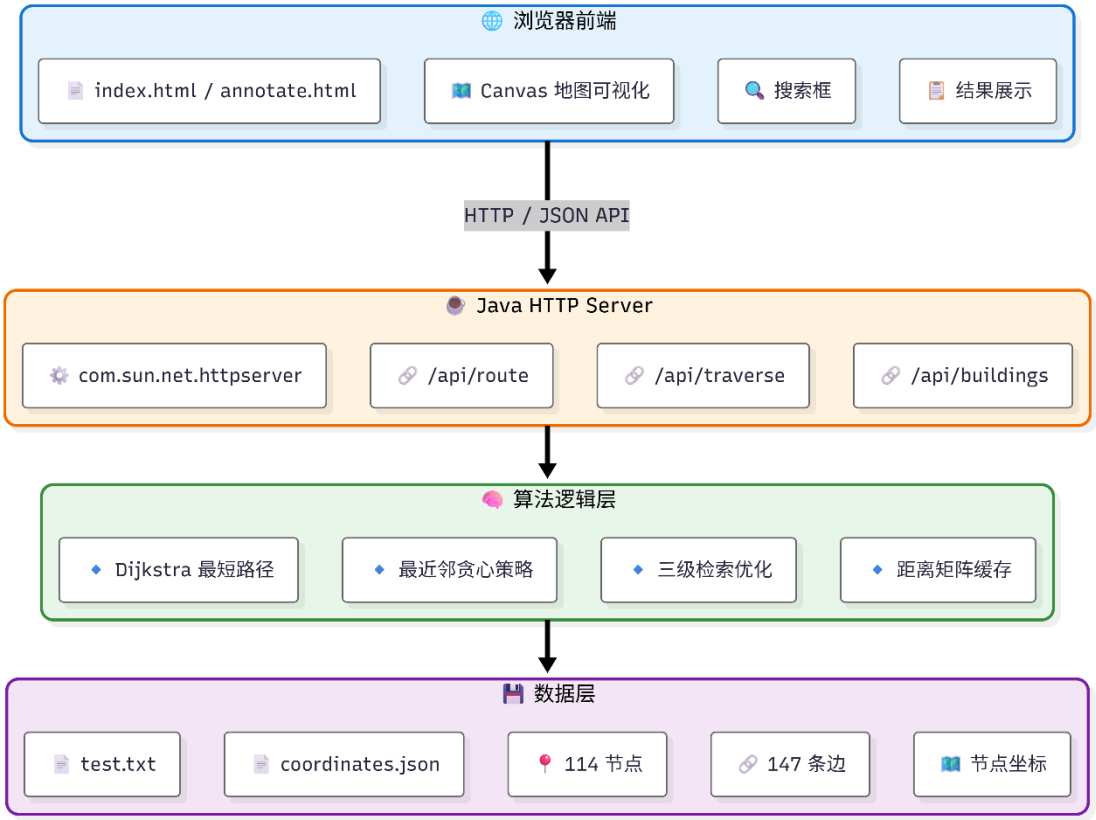
```
private synchronized void ensureDistMatrix() {
    if (distMatrix != null)
        return;
    for (String node : buildings.keySet())
        distMatrix.put(node, dijkstra(node).dist);
}
```

2.2.4 三级建筑检索

对输入关键字按优先级分三级匹配，结果合并后返回前 12 条：

优先级	匹配规则	示例
1（最高）	名称完全等于关键字	"图书馆" → 图书馆
2	名称以关键字为前缀	"图" → 图书馆南门
3	名称或描述包含关键字	"出入口" → 图书馆南门

2.2.5 系统整体架构



2.3 问题发现与解决

2.3.1 遍历功能响应慢

- 问题：初始实现每步重新运行 Dijkstra，114 次调用导致首次遍历有明显延迟。
- 解决：引入距离矩阵缓存，首次调用时预计算所有节点的单源最短路径并存储，后续遍历

直接查表，响应时间大幅缩短。

2.3.2 数据文件被非法输入破坏

- 问题：地图数据以逗号分隔的 CSV 格式存储，若建筑名称中包含逗号，写入后下次解析会产生错误分割，导致数据损坏。
- 解决：在 addBuilding()中增加输入校验，名称或描述含逗号时拒绝操作并给出提示。

2.3.3 程序路径硬编码，换电脑无法运行

- 问题：原始代码以 java 字符串硬编码数据目录，依赖程序从项目根目录启动，换台电脑或换目录运行即失效。
- 解决：实现三级路径自动检测（命令行参数 → 当前目录 → 从 class 文件位置向上查找），并提供跨平台启动脚本 run.sh（macOS/Linux）和 run.bat（Windows）。

三、程序复杂度分析

3.1 时间复杂度

设图中节点数 $V=114$，边数 $E=147$，需遍历建筑数 $N=83$。

功能	时间复杂度	说明
最短路径查询	$O((V+E)\log V)$	单次优先队列 Dijkstra
遍历首次调用	$O(V\cdot(V+E)\log V$	预计算全量距离矩阵，V 次 Dijkstra
遍历后续调用	$O(N^2)$	直接查距离矩阵，N 步每步 $O(N)$ 扫描
建筑检索	$O(V)$	遍历所有建筑做字符串匹配
增删建筑/道路	$O(V+E)$	CRUD + 缓存失效

对于本题规模，代入 $V=114, E=147, N=83$：单次最短路约 2257 次基本操作，几乎瞬时；遍历预计算约 38 万次操作，亦在毫秒级内完成。

3.2 空间复杂度

数据结构	空间复杂度	说明
邻接表 adjList	$O(V+E)$	存储所有节点及出边
建筑列表 buildings	$O(V)$	节点名称与描述
道路列表 roads	$O(E)$	用于持久化
距离矩阵 distMatrix	$O(V^2)$	每个节点到其余所有节点的距离
Dijkstra 单次运行	$O(V)$	距离数组与优先队列

主要空间开销为距离矩阵，$O(V^2)=114^2\approx 12996$ 个整数，约占 $116\,KB$，内存开销极小。

四、程序运行结果展示

4.1 最短路径查询

输入起点「学六公寓」，终点「校医院」，系统显示最短路径列表、总距离，地图上绘制橙色路线。

4.2 全建筑遍历

输入出发点「主楼东门」，系统输出建议访问 81 个建筑的顺序及总行程距离，地图绘制绿色路线。

4.3 建筑检索

搜索关键字「宿舍」，下拉框展示前 12 个匹配建筑，按精确 > 前缀 > 包含排序。

4.4 非法输入校验

在管理面板中新增名称为「测试,楼」的建筑，系统提示「名称不能包含逗号」，拒绝写入。

五、代码提交说明

项目	内容
代码文件	java/src/CampusNavSystem.java（后端主程序，含全部算法与 HTTP 服务） java/index.html（导航前端页面）

	java/annotate.html（坐标标注工具）
编译/运行环境	JDK 21（兼容 JDK 8+），无第三方依赖，使用 JDK 内置 com.sun.net.httpserver
编译命令	Javac -encoding UTF-8 -d out/production/java java/src/CampusNavSystem.java
运行方式	macOS/Linux：bash run.sh Windows：双击 run.bat 脚本自动编译并启动，浏览器访问 http://localhost:8080
输入数据说明	地图数据：java/test.txt（114 节点、147 条边） 节点坐标：java/coordinates.json（用于地图可视化）

六、实验过程记录

日期	时长	工作内容	备注
3 月 15 日	2h	分析题目要求，设计图的存储结构，选定邻接表方案	
3 月 17 日	3h	进行了两天的实际北邮地图数据采集，确定方案和图的基本建模，并上传数据集	
3 月 19 日	2h	完成了第一版的基本导航代码，并完成前端构建	
3 月 19 日	3h	进行测试后发现 Dijkstra 代码有优化空间，修改了原来的导航 bug，删去了部分边，并完成基本实验报告	
3 月 21 日	1h	优化了前端方案，增加了搜索模块，完成 debug	
3 月 22 日	2h	新增一个模糊化算法，可以在增删改道路时进行查找建筑，在寻路时支持模糊化搜索大批量道路	
3 月 23 日	1h	报告排版，润色，准备提交	

```
commit 0172f95116cf9c3b3bc18242d752a4375488b195 (HEAD ->
 main, origin/main)
Author: 780078268 <780078268@qq.com>
Date:   Sun Mar 22 12:23:42 2026 +0800

    修改了 md 文件，更好看

commit a40f0552467ea93ebf96f0b3351c78640f6a24c0
Author: 780078268 <780078268@qq.com>
Date:   Sun Mar 22 12:15:48 2026 +0800

    修改了检索的时候方式，新增了一个模糊化搜索算法，比如
可以同时搜索所有路口距离""

commit 615e61771daa952729fcfef7aa7d4e52e2922494
Author: 780078268 <780078268@qq.com>
Date:   Sat Mar 21 17:19:42 2026 +0800

    改进了代码，原来的管理页面中，想删除建筑和增加道路时
，无法进行搜索处理，必须把建筑名字每个字都敲对，现在多了
搜索模块

commit ff060514c04e10e4de391815dc2e99a16b711f0c
Author: 780078268 <780078268@qq.com>
Date:   Thu Mar 19 16:14:02 2026 +0800

    写报告

commit 5dfe50f809f18a55d77b3e1d55a46d544cceee8d
Author: 780078268 <780078268@qq.com>
Date:   Thu Mar 19 15:51:52 2026 +0800

    改掉了 bug，把 djstra 的代码优化了，减少复杂度

commit 16cf222fe53370c29a2eb95c0f28cd836452e42d
Author: 780078268 <780078268@qq.com>
Date:   Thu Mar 19 13:15:30 2026 +0800

    完成基础建模和导航代码

commit f0eb2087fe47071a7536c743779ad4826189408b
Author: 780078268 <780078268@qq.com>
Date:   Tue Mar 17 16:43:30 2026 +0800

    加入了校园地图的 txt 数据集

commit 02e1f0cb0d578e47753fda96678299bf46f03fd0
Author: 780078268 <780078268@qq.com>
Date:   Sun Mar 15 21:35:11 2026 +0800

    基本框架""
    ""

commit 7e87db94d37588a019f1ddbbc56ed7a50b17fb5f
Author: 780078268 <780078268@qq.com>
Date:   Sun Mar 15 21:17:13 2026 +0800

    初始化仓库

commit 2a017bf755d7f219663ef5ee85bce35a6c8d0141
Author: NiaoSuan <780078268@qq.com>
Date:   Sun Mar 15 21:21:28 2026 +0800

:
```

七、心得与总结

本次课程设计以北邮校园地图为背景，从零构建了一个完整的图论应用系统。在实践过程中有以下几点收获：

7.1 算法理解的深化

Dijkstra 算法在课堂上停留于理论层面，实际实现后才深刻体会到优先队列优化的意义——将朴素版本的 $O(V^2)$ 降低到 $O((V+E)\log V)$，对中大规模图的性能影响显著。同时，通过预计

算全量距离矩阵实现遍历提速，让我理解了「以空间换时间」这一经典权衡在工程中的具体体现。

7.2 数据结构选择的重要性

本题选用邻接表而非邻接矩阵，原因在于校园路网是稀疏图（$E = 147 \ll V^2 = 12996$），邻接表的空间与遍历效率均优于矩阵。这一选择在规模更大的地图数据上差距会更加明显。

7.3 系统设计的全局思维

一个完整的系统不仅仅是算法正确，还需要考虑数据完整性（逗号校验）、跨平台兼容性（路径自动检测、跨平台启动脚本）和用户体验（实时搜索、回车键支持、地图可视化）。这些工程细节虽不涉及核心算法，却决定了系统在真实场景中是否可用。

7.4 创新点

系统在遍历算法上引入了惰性距离矩阵缓存——首次调用时一次性预计算，后续调用直接复用，且在图结构变更时自动失效重建，兼顾了性能与正确性。前端采用 Canvas 实时绘制路径节点，辅以坐标标注工具，实现了完整的地图可视化闭环。